

S BHANU PRAKASH

Cybersecurity Enthusiast — Ethical Hacking & VAPT Practice

Web Application Reconnaissance & Content Discovery Report

Directory Enumeration and Fingerprinting — bWAPP Training Web Server

Target	10.232.2.58 (bWAPP training web server, isolated lab network)
Engagement Type	Web application reconnaissance & directory/content enumeration
Target Scope	http://10.232.2.58/ (bWAPP intentionally vulnerable app)
Tools Used	ffuf v2.1.0-dev, curl, Kali Linux
Date	17 July 2026
Prepared By	S Bhanu Prakash

Note: This assessment was carried out in an isolated personal/virtual lab environment against intentionally vulnerable practice systems, as part of independent cybersecurity skill-building. No production systems, third-party assets, or real organizations were involved or affected.

1. Executive Summary

This report documents a reconnaissance and content-discovery exercise I carried out against 10.232.2.58, an internal lab target running the bWAPP (buggy Web Application) training platform. The objective was to identify exposed directories, files, applications, and service-fingerprint information using automated fuzzing and manual verification, following a standard black-box web application testing methodology.

The assessment was carried out entirely using the command-line fuzzing tool ffuf together with curl for manual verification. A total of eight (8) findings were identified, ranging from Informational to High severity. The most significant issue is the presence of an end-of-life web/application stack (Apache 2.2.8, PHP 5.2.4, OpenSSL 0.9.8g), which is no longer supported or patched and is affected by numerous publicly documented vulnerabilities.

No exploitation, credential brute-forcing, or data modification was attempted — the scope was limited to unauthenticated reconnaissance and content discovery.

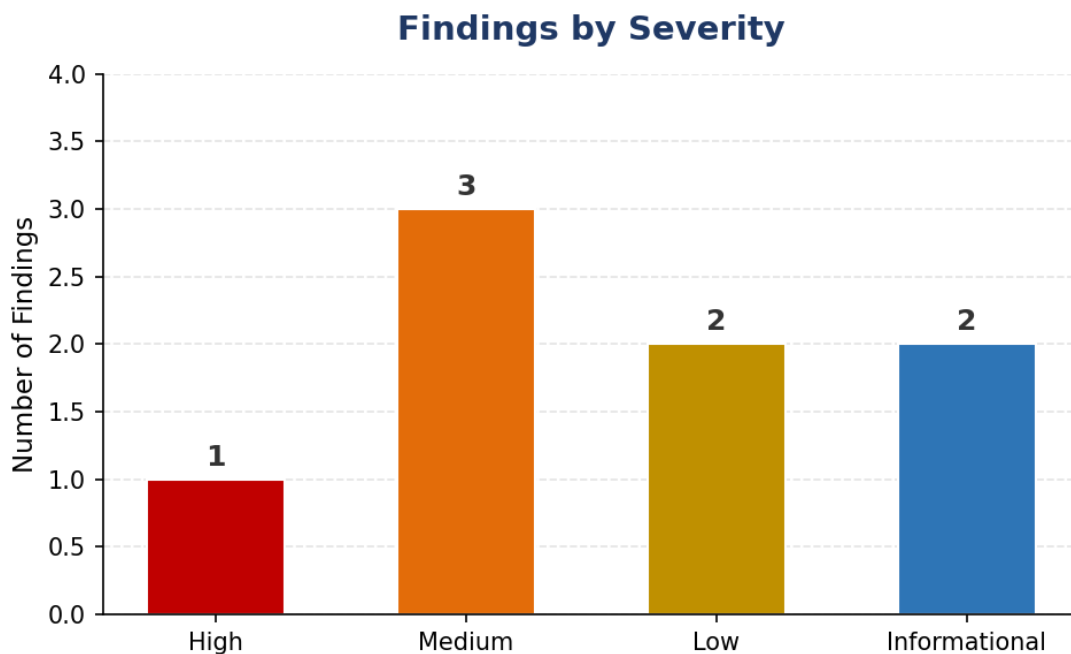


Figure 1: Distribution of findings by severity.

2. Scope & Objectives

2.1 Scope

- Target host: 10.232.2.58 (HTTP, port 80)
- In-scope activities: directory/file enumeration, HTTP header/banner analysis, service fingerprinting
- Out of scope: authenticated testing, exploitation of discovered vulnerabilities, denial-of-service testing, and any host other than 10.232.2.58

2.2 Objectives

- Identify hidden or unlinked directories and files exposed by the web server
- Fingerprint the underlying server, application, and framework versions
- Assess the WebDAV and phpMyAdmin components for unauthenticated exposure

- Provide risk-rated findings with clear remediation guidance

3. Methodology

The assessment followed a standard black-box content-discovery methodology, consistent with OWASP Web Security Testing Guide (WSTG-INFO series) practices:

- Reconnaissance — identify the target and confirm the web service is reachable on port 80.
- Content Discovery — brute-force directories and files using curated wordlists (wfuzz 'common.txt' and SecLists/DirBuster 'directory-list-2.3-medium.txt').
- Verification — manually confirm interesting hits using curl to inspect HTTP response headers and status codes.
- Targeted Fuzzing — perform focused fuzzing against specific discovered paths (e.g. /webdav/, /bWAPP/) to test for further hidden content and unusual server behaviour.
- Documentation — capture terminal evidence and raw tool output for every step performed.

All scanning used the default ffuf matcher configuration (200-299, 301, 302, 307, 401, 403, 405, 500), later widened to 'all' status codes where deeper insight into server behaviour was required.

4. Tools Used

Tool	Purpose
ffuf v2.1.0-dev	Fast web fuzzer for directory, file, and content discovery over HTTP
curl	Manual verification of discovered endpoints and response headers
Wordlists	wfuzz common.txt, DirBuster directory-list-2.3-medium.txt, wfuzz http_methods.txt, wfuzz common_pass.txt
Operating System	Kali Linux (attacker workstation)

5. Attack Surface Overview

The chart below shows the volume of requests sent across each phase of the engagement, from the initial common-wordlist sweep through to the deeper WebDAV-focused fuzzing sessions.

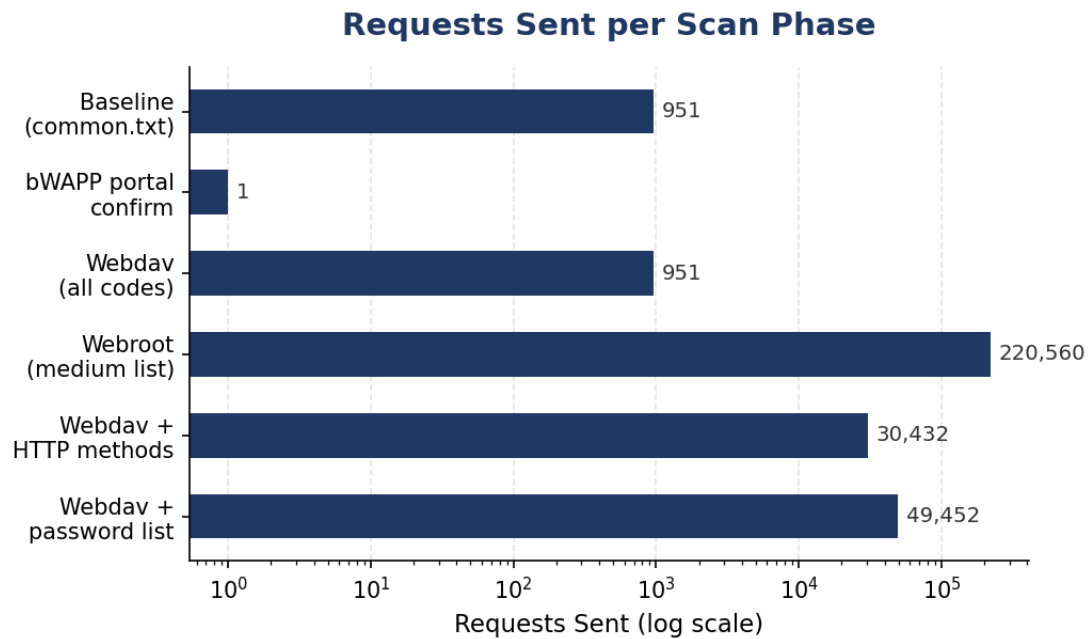


Figure 2: Requests sent per scan phase (log scale).

The medium-sized DirBuster wordlist scan against the webroot (220,560 requests) produced the widest coverage:

HTTP Status Code Distribution (Medium Wordlist Scan — 220,560 Requests)

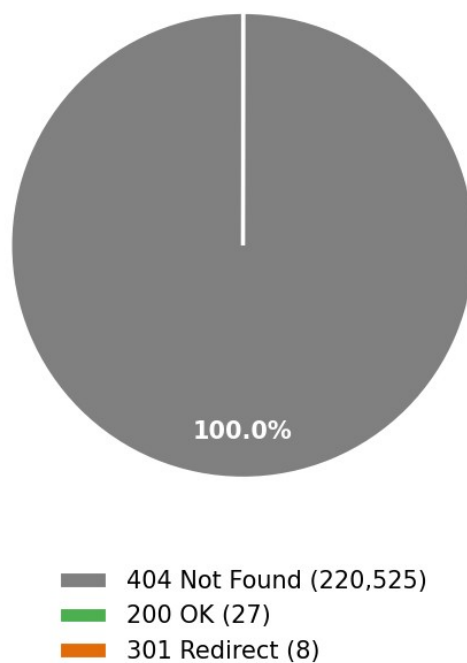


Figure 3: HTTP status code distribution across the medium wordlist scan.

6. Summary of Findings

#	Finding	Severity	Component / Location
F-01	Outdated / End-of-Life Software Stack (Apache 2.2.8, PHP 5.2.4, OpenSSL 0.9.8g)	High	http://10.232.2.58/
F-02	phpMyAdmin Interface Publicly Exposed	Medium	/phpmyadmin/
F-03	WebDAV Endpoint With Anomalous Response Behaviour	Medium	/webdav/
F-04	Intentionally Vulnerable Training Application (bWAPP) Reachable	Medium	/bWAPP/portal.php
F-05	Multiple Third-Party Applications Enumerable (Drupal, SQLite Manager)	Low	http://10.232.2.58/
F-06	Verbose Directory/File Enumeration Possible With No Rate Limiting	Low	http://10.232.2.58/
F-07	Apache mod_status Endpoint Accessible	Informational	/server-status
F-08	Version & Build Files Disclosed (README, INSTALL, release_notes)	Informational	http://10.232.2.58/

7. Detailed Findings

[HIGH] F-01 — Outdated / End-of-Life Software Stack

Affected: <http://10.232.2.58/>

HTTP response headers disclose a software stack that reached end-of-life several years ago: Apache 2.2.8 (Ubuntu), mod_fastcgi/2.4.6, PHP 5.2.4-2ubuntu5 (Suhosin-Patch), mod_ssl/2.2.8, and OpenSSL 0.9.8g. None of these receive security patches — PHP 5.2 has been unsupported since 2011, and OpenSSL 0.9.8g predates fixes for numerous well-known SSL/TLS vulnerabilities.

Impact: running unsupported software leaves the target exposed to publicly known, unpatched vulnerabilities that can lead to remote code execution, information disclosure, or full server compromise — the most severe structural issue identified.

Recommendation

- Upgrade Apache, PHP, and OpenSSL to currently supported, actively patched versions.
- Where legacy applications require an old PHP version for compatibility, isolate the host on a segmented, non-routable lab network only.
- Disable or mask verbose version banners (ServerTokens, expose_php directives).

```
(kali㉿kali)-[~]
└─$ curl -sIL http://10.232.2.58/phpmyadmin/
HTTP/1.1 200 OK
Date: Fri, 17 Jul 2026 14:44:30 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch mod_ssl/2.2.8 OpenSSL/0.9.8g
X-Powered-By: PHP/5.2.4-2ubuntu5
Set-Cookie: pmaCookieVer=4; expires=Sun, 16-Aug-2026 14:44:30 GMT; path=/phpmyadmin/; httponly
Set-Cookie: phpMyAdmin=vgd0V8h-V9oi4IICWfAwe2iuHnc; path=/phpmyadmin/; HttpOnly
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: private, max-age=10800, pre-check=10800
Last-Modified: Sat, 08 Dec 2007 12:47:13 GMT
Set-Cookie: pma_fontsize=82%25; expires=Sun, 16-Aug-2026 14:44:30 GMT; path=/phpmyadmin/; httponly
Set-Cookie: pma_mcrypt_iv=qGU9h0PbFho3D; expires=Sun, 16-Aug-2026 14:44:30 GMT; path=/phpmyadmin/; httponly
Content-Type: text/html; charset=utf-8
```

Figure 4: curl -sIL response headers disclosing Apache 2.2.8, PHP 5.2.4-2ubuntu5, and OpenSSL 0.9.8g.

[MEDIUM] F-02 — phpMyAdmin Interface Publicly Exposed

Affected: <http://10.232.2.58/phpmyadmin/>

Directory enumeration with ffuf identified a publicly reachable phpMyAdmin installation, returning an HTTP 301 redirect and subsequently an HTTP 200 response with a valid phpMyAdmin session cookie.

Impact: an exposed database-administration front end significantly increases the attack surface. If weak, default, or reused credentials are configured on the backend MySQL service, this interface could be leveraged for full database compromise.

Recommendation

- Restrict access to /phpmyadmin/ to trusted management IP ranges or place behind VPN / HTTP basic-auth.
- Ensure strong, unique credentials on the MySQL backend and keep phpMyAdmin fully patched.
- Consider renaming or removing the default path in production-like environments.

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ffuf -u http://10.232.2.58/FUZZ -w /usr/share/wordlists/wfuzz/general/common.txt

:: Method      : GET
:: URL         : http://10.232.2.58/FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/wfuzz/general/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

index          [Status: 200, Size: 45, Words: 2, Lines: 2, Duration: 3ms]
phpmyadmin     [Status: 301, Size: 405, Words: 27, Lines: 10, Duration: 6ms]
webdav         [Status: 301, Size: 401, Words: 27, Lines: 10, Duration: 8ms]
:: Progress: [951/951] :: Job [1/1] :: 36 req/sec :: Duration: [0:00:05] :: Errors: 0 ::

```

Figure 5: ffuf identifying /phpmyadmin/ and /webdav/ with HTTP 301 responses.

[MEDIUM] F-03 — WebDAV Endpoint With Anomalous Response Behaviour

Affected: <http://10.232.2.58/webdav/>

The Server header confirms the DAV/2 Apache module is loaded, and /webdav/ was confirmed reachable (HTTP 301) during the initial sweep. Follow-up fuzzing with the 'all' status-code matcher, and later a combined wordlist (49,452 requests), showed the endpoint returns an identical HTTP 200 response for effectively every path/word combination tested — a catch-all response rather than standard WebDAV/Apache behaviour.

Impact: an always-200 response can mask the true presence or absence of files, hindering both defenders and automated scanners from reliably mapping directory content. Depending on backend configuration, exposed WebDAV can also permit unauthenticated file upload/PUT operations if method restrictions are not enforced.

Recommendation

- Manually verify WebDAV method handling (OPTIONS, PUT, DELETE, PROPFIND) with authenticated and unauthenticated sessions.
- If WebDAV is not required, disable the mod_dav/mod_dav_fs modules entirely.
- If required, enforce authentication on all WebDAV verbs and restrict source IPs.

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali) [~]
$ ffuf -u http://10.232.2.58/webdav/FUZZ -w /usr/share/wordlists/wfuzz/general/common.txt -mc all -c

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.232.2.58/webdav/FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/wfuzz/general/common.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher      : Response status: all

HTML Injection - Reflected (Current URL)
1 ML Injection - Reflected (Current URL) [Status: 404, Size: 373, Words: 31, Lines: 10, Duration: 23ms]
01 ML Injection - Stored (Blog) [Status: 404, Size: 374, Words: 31, Lines: 10, Duration: 24ms]
10 Time Injection [Status: 404, Size: 374, Words: 31, Lines: 10, Duration: 25ms]
2003 Injection (Search) [Status: 404, Size: 376, Words: 31, Lines: 10, Duration: 24ms]
123 Header Injection (SMTP) [Status: 404, Size: 375, Words: 31, Lines: 10, Duration: 26ms]
2 [Status: 404, Size: 373, Words: 31, Lines: 10, Duration: 26ms]
administration [Status: 404, Size: 386, Words: 31, Lines: 10, Duration: 27ms]
administrator [Status: 404, Size: 385, Words: 31, Lines: 10, Duration: 2ms]

```

Figure 6: ffuf against /webdav/FUZZ with matcher set to 'all', surfacing standard 404 responses for the common wordlist.

```

kali@kali: ~
Session Actions Edit View Help

:: Progress: [30432/30432] :: Job [1/1] :: 4000 req/sec :: Duration: [0:00:09] :: Errors: 0 ::

(kali@kali) [~]
$ ffuf -u http://10.232.2.58/webdav/FUZZ -w /usr/share/wordlists/wfuzz/general/common.txt -w /usr/share/wordlists/wfuzz/others/common_pass.txt

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.232.2.58/webdav/FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/wfuzz/general/common.txt
:: Wordlist     : FUZZ: /usr/share/wordlists/wfuzz/others/common_pass.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500

sim [Status: 200, Size: 2247, Words: 131, Lines: 21, Duration: 21ms]
* FUZZ:
* FUZZ:
through [Status: 200, Size: 2247, Words: 131, Lines: 21, Duration: 23ms]
* FUZZ:
* FUZZ:
real [Status: 200, Size: 2247, Words: 131, Lines: 21, Duration: 25ms]
* FUZZ:
* FUZZ:
E [Status: 200, Size: 2247, Words: 131, Lines: 21, Duration: 25ms]
* FUZZ:
* FUZZ:
inst [Status: 200, Size: 2247, Words: 131, Lines: 21, Duration: 25ms]
* FUZZ:
* FUZZ:

```

Figure 7: Extended fuzzing (49,452 requests) showing a constant HTTP 200 response for every combination — the anomaly behind F-03.

[MEDIUM] F-04 — Intentionally Vulnerable Training Application (bWAPP) Reachable

Affected: <http://10.232.2.58/bWAPP/portal.php>

The target was confirmed to be running bWAPP ('buggy Web Application'), an intentionally vulnerable PHP/MySQL training application. The login portal at /bWAPP/portal.php returned an HTTP 302 redirect, confirming the application

Impact: bWAPP is deliberately riddled with vulnerabilities (SQLi, XSS, command injection, file inclusion, and more) by design for training purposes. If this host were ever reachable from an untrusted network, every one of these built-in vulnerabilities would be trivially exploitable by any user reaching the application.

Recommendation

- Confirm the host is strictly confined to an isolated training/lab network segment with no route to production systems or the public internet.
- Apply firewall rules limiting inbound access to authorised student/tester IP ranges only.
- Tear down or snapshot-reset the environment after each training exercise.

[LOW] F-05 — Multiple Third-Party Applications Enumerable

Affected: <http://10.232.2.58/>

In addition to bWAPP and phpMyAdmin, the medium-wordlist DirBuster scan (220,560 requests) revealed a 'drupal' directory and a 'sqlite' directory (SQLite Manager), each returning HTTP 301 redirects — as shown earlier in Figure 5.

Impact: a single host running multiple independent web applications increases the overall attack surface; a vulnerability in any one component could be used to pivot to, or otherwise affect, the others.

Recommendation

- Where possible, isolate each application on its own host or container.
- Inventory and version-check every co-hosted application, applying the same patch-management standard used elsewhere.

[LOW] F-06 — Verbose Directory/File Enumeration Possible With No Rate Limiting

Across the engagement, over 300,000 fuzzing requests were sent to the target (951 + 220,560 + 951 + 30,432 + 49,452 requests across five separate ffuf sessions) without the target ever throttling, blocking, or otherwise rate-limiting the source. Sustained throughput of up to ~4,500 requests/second was observed with zero errors reported by the tool.

Impact: the absence of any request throttling, WAF, or IDS/IPS response makes the target trivial to fully enumerate and considerably eases reconnaissance for an attacker, as well as any brute-force attempts against authentication endpoints.

Recommendation

- Deploy a Web Application Firewall (WAF) or reverse proxy with rate-limiting rules.
- Monitor and alert on high-volume 404 patterns indicative of directory brute-forcing.
- Consider fail2ban-style IP banning for sustained high-frequency scanning behaviour.

```

kali@kali: ~
Session Actions Edit View Help
wstats [Status: 404, Size: 378, Words: 31, Lines: 10, Duration: 4031ms]
web [Status: 404, Size: 375, Words: 31, Lines: 10, Duration: 4046ms]
:: Progress: [951/951] :: Job [1/1] :: 48 req/sec :: Duration: [0:00:04] :: Errors: 0 ::

(kali@kali)-[~]
$ ffuf -u http://10.232.2.58/webdav/FUZZ -w /usr/share/wordlists/wfuzz/general/common.txt -w /usr/share/wordlists/wfuzz/general/http_methods.txt

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.232.2.58/webdav/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/wfuzz/general/common.txt
:: Wordlist    : FUZZ: /usr/share/wordlists/wfuzz/general/http_methods.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

:: Progress: [30432/30432] :: Job [1/1] :: 4000 req/sec :: Duration: [0:00:09] :: Errors: 0 ::

(kali@kali)-[~]
$

```

Figure 9: A 30,432-request combined wordlist scan completing in 9 seconds at ~4,000 req/sec with zero errors — no rate limiting observed.

[INFORMATIONAL] F-07 — Apache mod_status Endpoint Accessible

Affected: <http://10.232.2.58/server-status>

The medium-wordlist scan identified `/server-status` returning HTTP 200 with a substantial response (15,756 bytes), indicating the Apache `mod_status` module is enabled and reachable without authentication.

Impact: `mod_status` can disclose internal information such as active worker processes, client IP addresses, and requested URLs, aiding an attacker in profiling server load, internal architecture, and user activity.

Recommendation

- Restrict `/server-status` to localhost or trusted management networks only, via Apache `<Location>` access controls.
- Disable `mod_status` entirely if not actively used for monitoring.

[INFORMATIONAL] F-08 — Version & Build Files Disclosed

Affected: <http://10.232.2.58/>

Standard framework build artefacts — `README`, `INSTALL`, and `release_notes` — were found directly accessible in the webroot, consistent with a default/unmodified Drupal installation. These files commonly disclose exact software version numbers.

Impact: precise version disclosure allows an attacker to quickly match the target against public vulnerability databases (CVE/NVD) without needing to fingerprint the application through slower, less reliable means.

Recommendation

- Remove or restrict access to default installation documentation files after deployment.
- Automate post-install hardening/cleanup as part of the deployment pipeline.

8. Command Log — Selected Evidence

Selected reconnaissance commands executed during the engagement, in chronological order:

Step	Command
Baseline scan	<code>ffuf -u http://10.232.2.58/FUZZ -w /usr/share/wordlists/wfuzz/general/common.txt</code>
Header verification	<code>curl -sIL http://10.232.2.58/phpmyadmin/</code>
Application confirmation	<code>ffuf -u http://10.232.2.58/bWAPP/portal.php -w /usr/share/wordlists/wfuzz/general/common.txt</code>
Deep WebDAV fuzzing	<code>ffuf -u http://10.232.2.58/webdav/FUZZ -w ../common.txt -mc all -c</code>
Multi-wordlist fuzzing	<code>ffuf -u http://10.232.2.58/webdav/FUZZ -w common.txt -w http_methods.txt</code>
Extended password-list fuzzing	<code>ffuf -u http://10.232.2.58/webdav/FUZZ -w common.txt -w common_pass.txt</code>

```

:: Progress: [1/1] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
(kali@kali)-[~]
$ ffuf -u http://10.232.2.58/webdav/FUZZ -w /usr/share/wordlists/wfuzz/general/common.txt -mc 200,301,302,403 -c

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.232.2.58/webdav/FUZZ
:: Wordlist     : FUZZ: /usr/share/wordlists/wfuzz/general/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher     : Response status: 200,301,302,403

:: Progress: [951/951] :: Job [1/1] :: 45 req/sec :: Duration: [0:00:04] :: Errors: 0 ::

```

Figure 10: Baseline ffuf command against the webroot using the common wordlist.

9. Risk Rating Reference

Rating	Meaning
High	Directly exploitable or represents unsupported/unpatched core infrastructure; requires urgent remediation.
Medium	Exposes sensitive functionality or applications that materially widen the attack surface; remediate in the near term.
Low	Increases attack surface or aids attacker reconnaissance but has no direct, immediate exploit path.
Informational	Observations that do not represent a vulnerability on their own but are useful for hardening and

Rating	Meaning
	awareness.

10. Overall Risk Assessment

The overall risk posture of 10.232.2.58 is assessed as HIGH, driven primarily by the end-of-life software stack (F-01) combined with multiple exposed administrative and training interfaces (F-02, F-03, F-04) that would be severely impactful if this host were ever reachable outside a controlled lab environment.

As this target is a deliberately vulnerable bWAPP training instance, the findings are expected and confirm the lab environment is functioning and reachable as intended. The recommendations above are nonetheless treated as mandatory guidance for any equivalent host not intentionally provisioned for training.

11. Conclusion

This engagement enumerated the content, technology stack, and attack surface of 10.232.2.58 using ffuf and curl. Eight findings were documented and rated according to standard severity guidance. All activity was limited to unauthenticated reconnaissance; no exploitation or data access was performed.

What I practiced in this exercise

- Building and tuning ffuf fuzzing sessions (matcher configuration, multi-wordlist combinations, status-code filtering).
- Distinguishing genuine findings from scanner noise by manually verifying with curl.
- Recognizing anomalous server behaviour (a constant HTTP 200 response) as a signal worth deeper investigation rather than a dead end.
- Writing a full findings-based report with a risk rating scale and structured remediation guidance.